

Ingeniería del Software

Educación Media Superior

Modelado de sistemas

Mag. Lic. Eduardo Olivera

Modelado de sistemas

Según Ian Sommerville, el modelado de sistemas es una actividad esencial del proceso de ingeniería del software, ya que **permite representar un sistema** desde diferentes perspectivas con el fin de comprenderlo, analizarlo y comunicar su estructura y comportamiento **antes de su implementación** (Sommerville, 2016).

Por su parte, Pressman señala que los modelos constituyen una **representación simplificada de la realidad** que facilita la comprensión de sistemas complejos y permite reducir riesgos durante el desarrollo, ya que posibilitan la **validación temprana** de decisiones de diseño (Pressman, 2014).

En este marco, el modelado no es solo una técnica gráfica, sino una herramienta de análisis, abstracción y validación.

Modelado de sistemas

- Modelo Esencial
- UML (Lenguaje Unificado de Modelado)

Modelado de sistemas

Modelo Esencial

- Modelo Ambiental
- Modelo del Comportamiento
- Lista de Acontecimientos

UML

- Diagramas de Navegabilidad
- Diagramas de casos de uso
- Elementos básicos actores y relaciones
- Diagramas de clases
- Conceptos fundamentales: clases, atributos, métodos y relaciones (herencia, asociación, agregación, composición)
- Diagramas de secuencia (elementos y notaciones: Objetos, mensajes, activaciones y líneas de vida)
- Diagramas de actividad: actividades, decisiones, flujos de control y objetos.

Modelado de sistemas

Modelo Esencial

El **Modelo Esencial** es una etapa específica dentro del modelado de sistemas, centrada en describir el sistema considerando únicamente el dominio del problema y no la solución tecnológica.

Siguiendo a **Pressman (2014)**, en la fase de análisis el modelado debe enfocarse en representar el problema, evitando decisiones de diseño o implementación.

El Modelo Esencial:

- Define el entorno (Modelo Ambiental)
- Describe cómo responde el sistema (Modelo de Comportamiento)
- Enumera los eventos externos (Lista de Acontecimientos)

Su propósito es asegurar una comprensión completa del problema antes de avanzar al diseño detallado del sistema.

Modelado de sistemas

Modelo Ambiental

De acuerdo con **Sommerville (2016)**, los modelos de contexto (ambientales) muestran los límites del sistema y sus relaciones con otros sistemas o actores externos.

El Modelo Ambiental define:

- Límite del sistema
- Entidades externas
- Interacciones (flujos de información)

Modelado de sistemas

Modelo Ambiental

Función

Permite establecer claramente:

- Qué pertenece al sistema
- Qué está fuera del sistema
- Qué intercambios de información existen

Es el primer paso para delimitar correctamente el alcance del proyecto.

Modelado de sistemas

Modelo Ambiental

Límites del sistema

Según **Sommerville (2016)**, los modelos de contexto muestran los límites del sistema y sus relaciones con otros sistemas. El límite del sistema define qué funcionalidades y procesos forman parte del sistema en estudio y cuáles pertenecen al entorno externo.

En este sentido, el **límite del sistema** establece la frontera conceptual que separa:

- Lo que el sistema debe controlar o gestionar.
- Lo que permanece fuera de su responsabilidad.

Modelado de sistemas

Modelo Ambiental

Entidades externas

De acuerdo con **Sommerville (2016)**, en los modelos de contexto se identifican las entidades externas que interactúan con el sistema. Estas pueden ser personas, organizaciones u otros sistemas que intercambian información con él.

Por lo tanto, una **entidad externa** es cualquier elemento del entorno que:

- No forma parte del sistema.
- Interactúa con el sistema enviando o recibiendo información.

Ej: Personas, Organizaciones, APIs, Dispositivos de hardware (sensores, impresoras, etc).

Modelado de sistemas

Modelo Ambiental

Interacciones

Según **Pressman (2014)**, durante el análisis del sistema se deben identificar los flujos de información entre el sistema y su entorno, ya que estos describen cómo el sistema responde a estímulos externos.

En el modelo ambiental, las **interacciones** representan:

- El intercambio de información entre el sistema y las entidades externas.
- Las entradas que recibe el sistema.
- Las salidas que produce como respuesta.

Modelado de sistemas

Modelo Ambiental

Para resolver un caso mediante el Modelo Ambiental, debemos ejecutar cuatro pasos:

1. Identificar qué debe hacer el sistema
2. Identificar entidades externas
3. Definir el alcance del sistema
4. Construir el Modelo Ambiental

Modelado de sistemas

Modelo Ambiental

1. Identificar qué debe hacer el sistema

Extraer del caso:

- Funcionalidades principales.
- Servicios que el sistema brinda.
- Respuestas que el sistema genera.

Esto surge del análisis del problema

Modelado de sistemas

Modelo Ambiental

2. Identificar entidades externas

Determinar:

- Personas que interactúan con el sistema.
- Organizaciones que reciben información.
- Otros sistemas externos (sensores, cámaras, impresoras, etc)

Modelado de sistemas

Modelo Ambiental

3. Definir el alcance del sistema

Establecer claramente:

Que Incluye:

- Qué responsabilidades son del sistema.

Que No incluye:

- Qué procesos pertenecen al entorno.
- Qué sistemas externos no forman parte del desarrollo.

Modelado de sistemas

Modelo Ambiental

4. Construir el Modelo Ambiental

Finalmente:

- Dibujar el límite del sistema.
- Colocar el sistema como unidad central.
- Ubicar entidades externas fuera.
- Representar flujos de información con dirección correcta.

Modelado de sistemas

Modelo Ambiental

Caso de Estudio - Sistema de Gestión de Biblioteca

Una institución educativa desea informatizar la gestión de su biblioteca mediante el desarrollo de un sistema que permita administrar préstamos de libros, devoluciones y el control de multas.

Actualmente, los procesos se realizan de forma manual, lo que genera demoras, errores en el registro y dificultades en la generación de reportes para la dirección.

La institución solicita el desarrollo de un sistema informático denominado:

Sistema de Gestión de Biblioteca

Modelado de sistemas

Modelo Ambiental

1. Qué debe hacer el sistema:

- Registrar préstamos de libros solicitados por los alumnos.
- Registrar devoluciones realizadas por los alumnos.
- Calcular multas en caso de retraso en la devolución.
- Generar un reporte mensual para la Dirección.

Modelado de sistemas

Modelo Ambiental

2. Entidades externas

El sistema interactúa con las siguientes entidades externas:

1. Alumno

- Solicita préstamos de libros.
- Recibe confirmación del préstamo realizado.

2. Bibliotecario

- Registra las devoluciones de libros.
- Recibe confirmación de que la devolución fue registrada correctamente.

Modelado de sistemas

Modelo Ambiental

2. Entidades externas

3. Dirección

- Recibe un reporte mensual con información estadística sobre préstamos y devoluciones.

4. Sistema de Pagos

- Recibe los datos correspondientes a multas generadas.
- Envía confirmación de pago al sistema una vez que la multa es abonada.

Modelado de sistemas

Modelo Ambiental

3. Alcance del Sistema

El sistema incluye:

- Registro de préstamos.
- Registro de devoluciones.
- Cálculo de multas.
- Generación de reportes.
- Envío de datos de multas al sistema de pagos.
- Recepción de confirmación de pago.

Modelado de sistemas

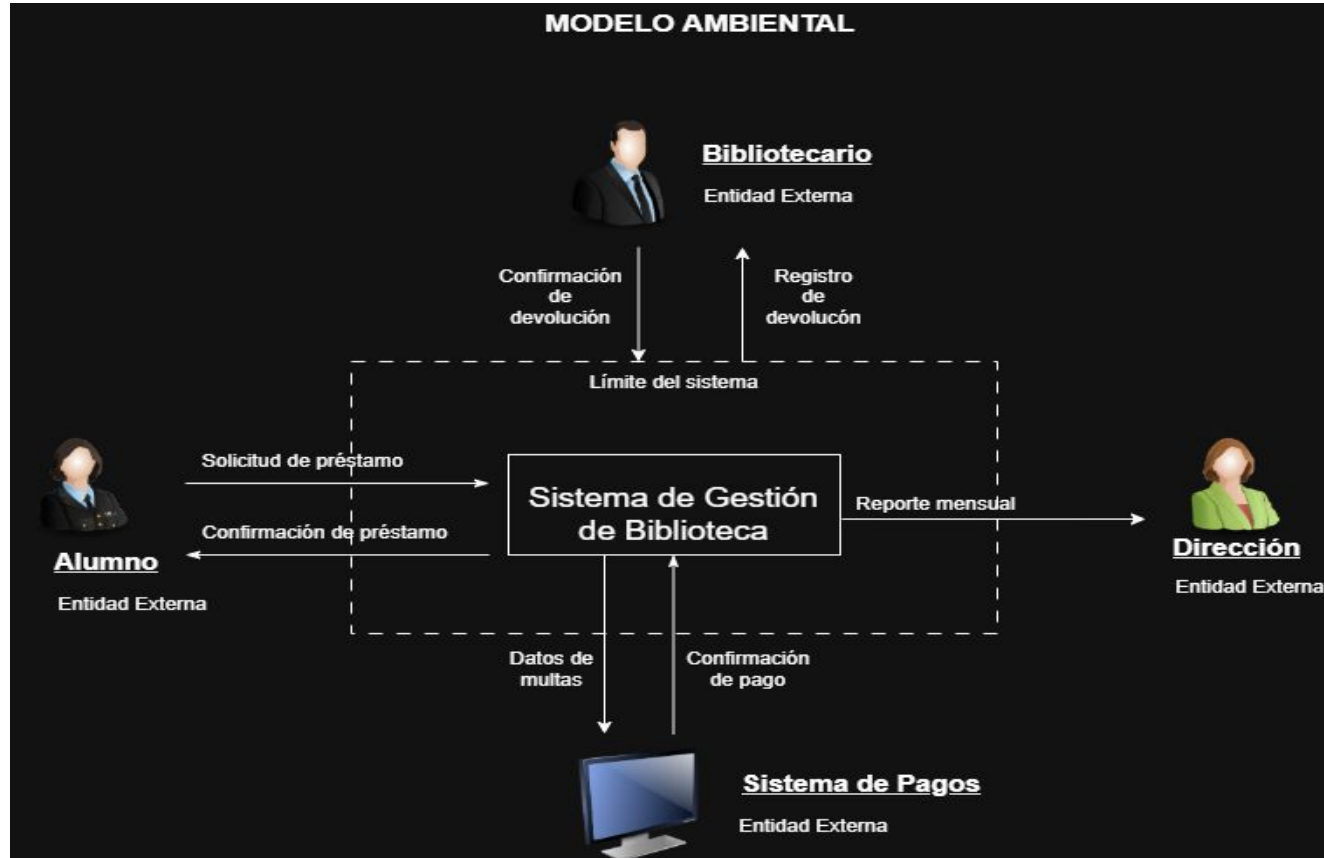
Modelo Ambiental

3. Alcance del Sistema

El sistema no incluye:

- Gestión académica.
- Gestión contable institucional.
- Procesamiento interno del sistema de pagos.
- Administración de usuarios externos.

Modelado de sistemas



Modelado de sistemas

Modelo de Comportamiento

Según **Sommerville (2016)**, los modelos de comportamiento describen cómo el sistema reacciona ante eventos y cómo se producen las transiciones entre estados.

El **Modelo de Comportamiento** representa:

- Los eventos que recibe el sistema
- Las respuestas que genera
- La transformación de entradas en salidas

Modelado de sistemas

Modelo de Comportamiento

Función

Permite comprender:

- La dinámica del sistema
- El flujo lógico de procesamiento
- La respuesta ante estímulos externos

Se enfoca en **cómo actúa el sistema**, no en su estructura interna.

Modelado de sistemas

Modelo de Comportamiento

Para modelar el Comportamiento del sistema, debemos:

1. Identificar eventos externos
2. Definir la respuesta del sistema
3. Determinar estados (si aplica)
4. Representar el modelo gráficamente

Modelado de sistemas

Modelo de Comportamiento

1. Identificar eventos externos

Se parte de la Lista de Acontecimientos.

Ejemplo:

- Solicitud de préstamo
- Registro de devolución
- Confirmación de pago

Cada evento debe provocar una respuesta del sistema.

Modelado de sistemas

Modelo de Comportamiento

2. Definir la respuesta del sistema

Para cada evento:

Evento → ¿Qué hace el sistema?

Ejemplo: solicitud de préstamo →

- Verifica disponibilidad
- Registra préstamo
- Genera confirmación

Modelado de sistemas

Modelo de Comportamiento

3. Determinar estados (si aplica)

En sistemas donde el estado cambia:

Libro (Disponible, Prestado, Reservado)

Multa (Pendiente, Pagada)

Esto permite modelar comportamiento basado en estados.

Modelado de sistemas

Modelo de Comportamiento

4. Representar el modelo gráficamente

Puedes optar por uno de estos modelos:

- a. Diagrama de estados: para modelar cambios de estado del sistema o de una entidad.

Ejemplo: Disponible → Prestado → Devuelto

- b. Diagrama de Actividad: Para modelar flujo de procesamiento interno.

Ejemplo: Solicitud de préstamo → Verificar disponibilidad → Registrar préstamo → Emitir confirmación

Modelado de sistemas

Modelo de Comportamiento

Continuando con el caso de ejemplo, Sistema de Gestión de Biblioteca, ahora lo abordaremos desde la perspectiva del Modelo de Comportamiento.

1. Eventos externos:

Partimos del Modelo Ambiental y de la Lista de Acontecimientos.

Eventos que llegan al sistema:

1. Solicitud de préstamo (Alumno)
2. Registro de devolución (Bibliotecario)
3. Confirmación de pago (Sistema de Pagos)

Evento que genera el sistema:

4. Generación de reporte mensual (proceso periódico)

Modelado de sistemas

Modelo de Comportamiento

2. Definir la respuesta del sistema: ahora describimos qué hace el sistema ante cada evento.

Solicitud de préstamo

Respuesta del sistema:

1. Verifica disponibilidad del libro.
2. Si está disponible:
 - Registra el préstamo.
 - Cambia estado del libro a “Prestado”.
 - Envía confirmación al alumno.
3. Si no está disponible:
 - Informa indisponibilidad.

Modelado de sistemas

Modelo de Comportamiento

2. Definir la respuesta del sistema: ahora describimos qué hace el sistema ante cada evento.

Registro de devolución

Respuesta del sistema:

1. Registra la devolución.
2. Cambia estado del libro a “Disponible”.
3. Verifica si existe retraso.
4. Si hay retraso:
 - Calcula multa.
 - Genera datos de multa.
 - Envía datos al Sistema de Pagos.
5. Envía confirmación al bibliotecario.

Modelado de sistemas

Modelo de Comportamiento

2. Definir la respuesta del sistema: ahora describimos qué hace el sistema ante cada evento.

Confirmación de pago

Respuesta del sistema:

1. Actualiza estado de la multa a “Pagada”.
2. Registra fecha de pago.
3. Confirma recepción del pago.

Modelado de sistemas

Modelo de Comportamiento

2. Definir la respuesta del sistema: ahora describimos qué hace el sistema ante cada evento.

Generación de reporte mensual

Respuesta del sistema:

1. Recolecta datos de préstamos.
2. Recolecta datos de devoluciones.
3. Genera informe estadístico.
4. Envía reporte a Dirección.

Modelado de sistemas

Modelo de Comportamiento

3. Determinar estados: Identificamos entidades que cambian de estado.

Entidad: Libro

Estados posibles:

- Disponible
- Prestado

Transiciones:

Disponible → Prestado (cuando se registra préstamo)

Prestado → Disponible (cuando se registra devolución)

Modelado de sistemas

Modelo de Comportamiento

3. Determinar estados: Identificamos entidades que cambian de estado.

Entidad: Multa

Estados posibles:

- Pendiente
- Pagada

Transiciones:

Pendiente → Pagada (cuando se recibe confirmación de pago)

Modelado de sistemas

Modelo de Comportamiento

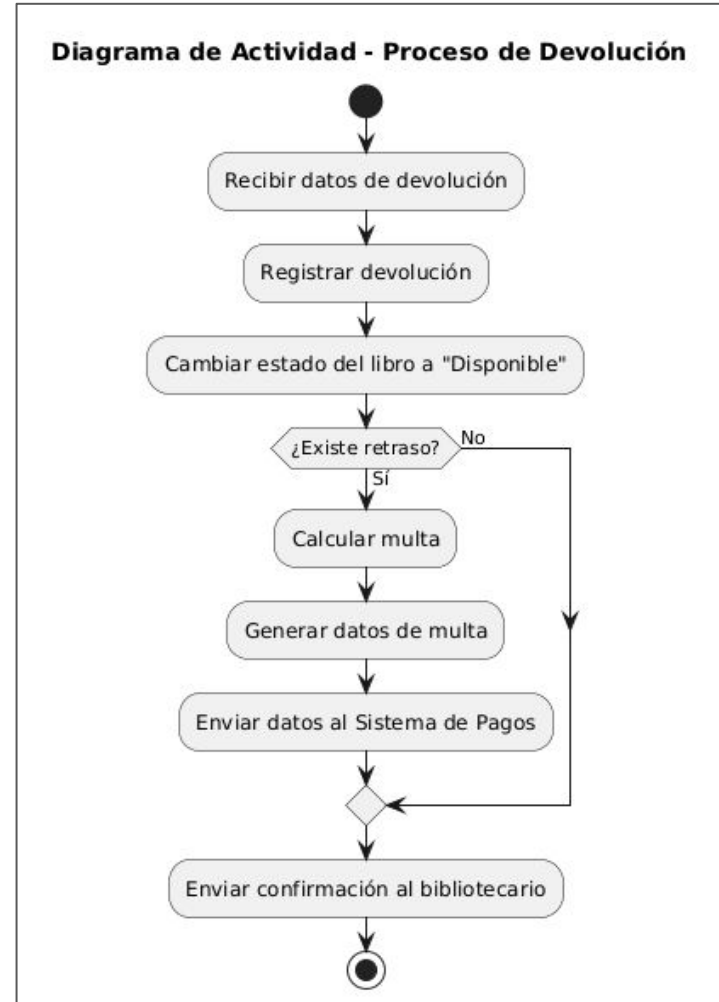
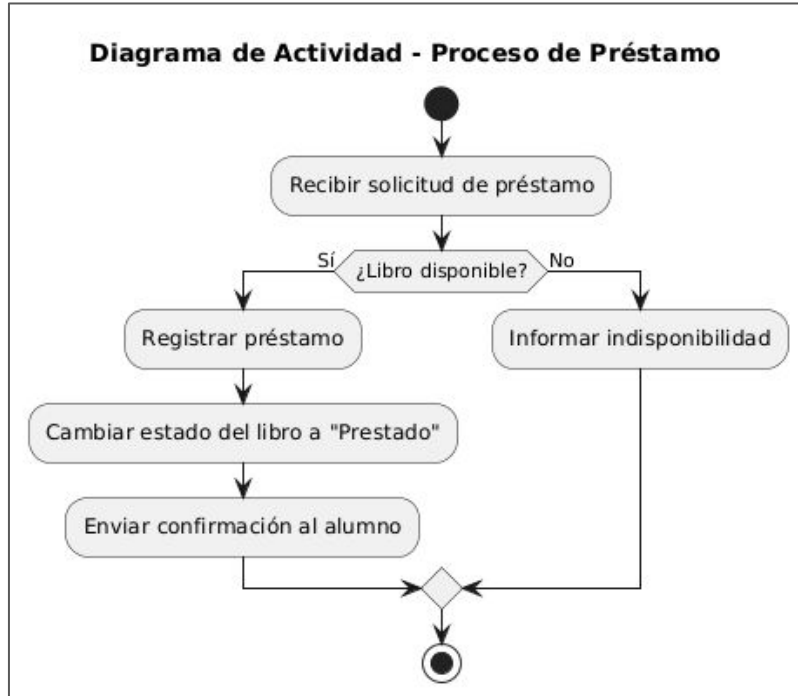
4. Representación gráfica:

Para este caso conviene utilizar:

- Diagrama de Actividad (flujo del proceso)
- Diagrama de Estados (para Libro y Multa)

Modelado de sistemas

Modelo de Comportamiento



Modelado de sistemas

Modelo de Comportamiento

Diagrama de Actividad - Confirmación de Pago

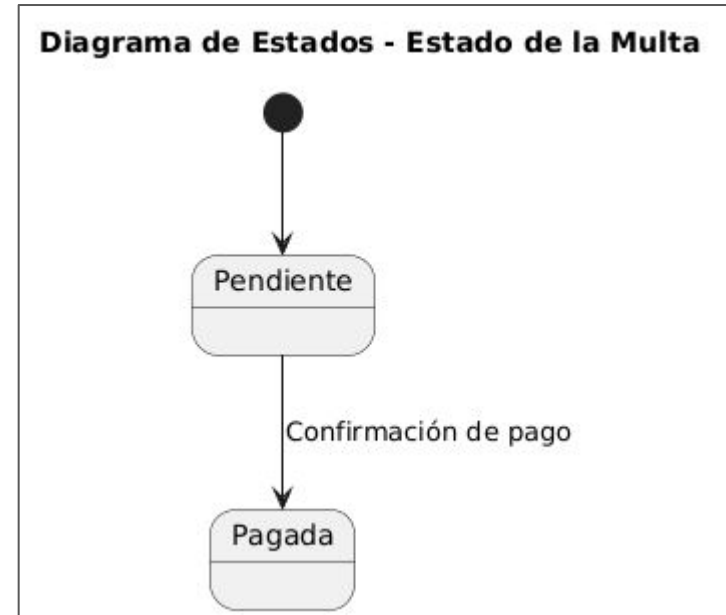


Diagrama de Actividad - Generación de Reporte Mensual



Modelado de sistemas

Modelo de Comportamiento



Modelado de sistemas

Modelo de Comportamiento

Resumen

El Modelo de Comportamiento toma los eventos identificados en el Modelo Ambiental (o en la Lista de Acontecimientos) y modela cómo el sistema responde a ellos, representando su flujo de actividades y/o los cambios de estado que provocan.

Modelado de sistemas

Lista de acontecimientos

Consiste en el registro sistemático de todos los eventos externos que requieren una respuesta del sistema (Pressman, 2014).

La Lista de Acontecimientos enumera:

- Evento externo (se originan fuera del sistema)
- Dispara procesamiento interno
- Genera una respuesta del sistema

Modelado de sistemas

Lista de acontecimientos

Función

- Sirve como puente entre requerimientos y modelado
- Garantiza que ningún evento relevante quede sin contemplar
- Facilita la trazabilidad con los casos de uso

Modelado de sistemas

Lista de acontecimientos

Continuando con el caso del Sistema de Gestión de Biblioteca, resolvemos este punto:

Nº	Acontecimiento (Evento Externo)	Origen	Respuesta del Sistema
1	El <u>alumno</u> solicita un préstamo	Alumno	Verifica disponibilidad, registra préstamo si corresponde y envía confirmación
2	El <u>bibliotecario</u> registra una devolución	Bibliotecario	Registra devolución, cambia estado del libro, verifica retraso y calcula multa si corresponde
3	El <u>sistema de pagos</u> confirma el pago de una multa	Sistema de Pagos	Actualiza estado de la multa a “Pagada” y registra la fecha de pago
4	Se alcanza el cierre del período mensual	Evento temporal (Calendario)	Genera reporte mensual y lo envía a la Dirección

UML (Lenguaje Unificado de Modelado)

Clasificación de Diagramas

Definición

Según **Sommerville (2016)**, UML es un lenguaje de modelado gráfico estandarizado que proporciona un conjunto de diagramas para especificar, visualizar, construir y documentar los artefactos de un sistema de software.

Por su parte, **Pressman (2014)** sostiene que UML ofrece diferentes tipos de diagramas que permiten representar distintas perspectivas del sistema: estructural, conductual y de interacción.

UML (Lenguaje Unificado de Modelado)

Categorías de diagramas UML

Según **Pressman (2014)** y **Sommerville (2016)**, los diagramas UML se clasifican en:

2.1 Diagramas estructurales

Representan la estructura estática del sistema: clases, objetos, componentes y sus relaciones.

2.2 Diagramas de comportamiento

Modelan el comportamiento dinámico del sistema, mostrando cómo responde ante eventos.

2.3 Diagramas de interacción

Subtipo de los diagramas de comportamiento que enfatizan el intercambio de mensajes entre objetos.

Diagrama de Navegabilidad

Definición

Aunque UML no define formalmente un “diagrama de navegabilidad” como categoría estándar, **Pressman (2014)** describe los modelos de navegación como representaciones que muestran cómo los usuarios se desplazan entre distintas vistas o funcionalidades de un sistema.

En aplicaciones web o con interfaz gráfica, representan la estructura de pantallas y sus vínculos, siendo el Diagrama de Navegabilidad es una herramienta complementaria.

Diagrama de Navegabilidad

Categorías

Puede representarse como:

- Navegación **jerárquica**: organización en forma de árbol.
- Navegación **en red**: múltiples caminos entre nodos.
- Navegación **secuencial**: flujo lineal obligatorio.

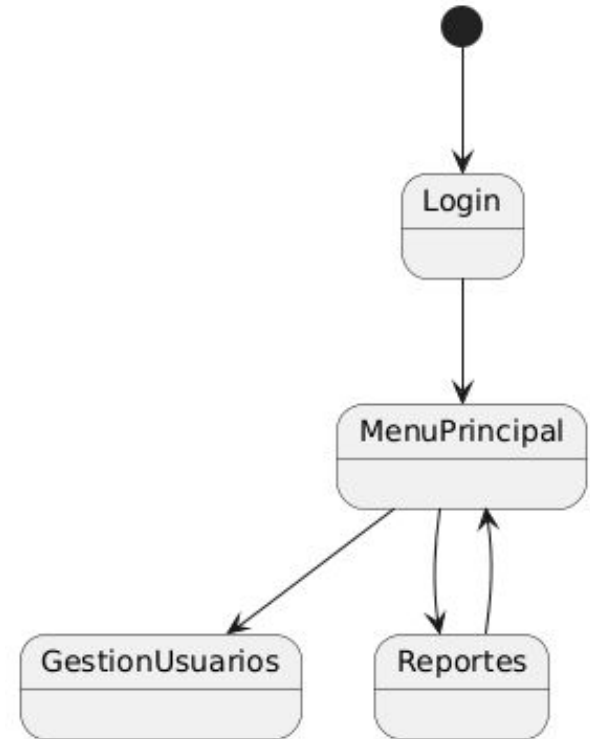


Diagrama de Navegabilidad

Para nuestro caso de Biblioteca, tenemos...

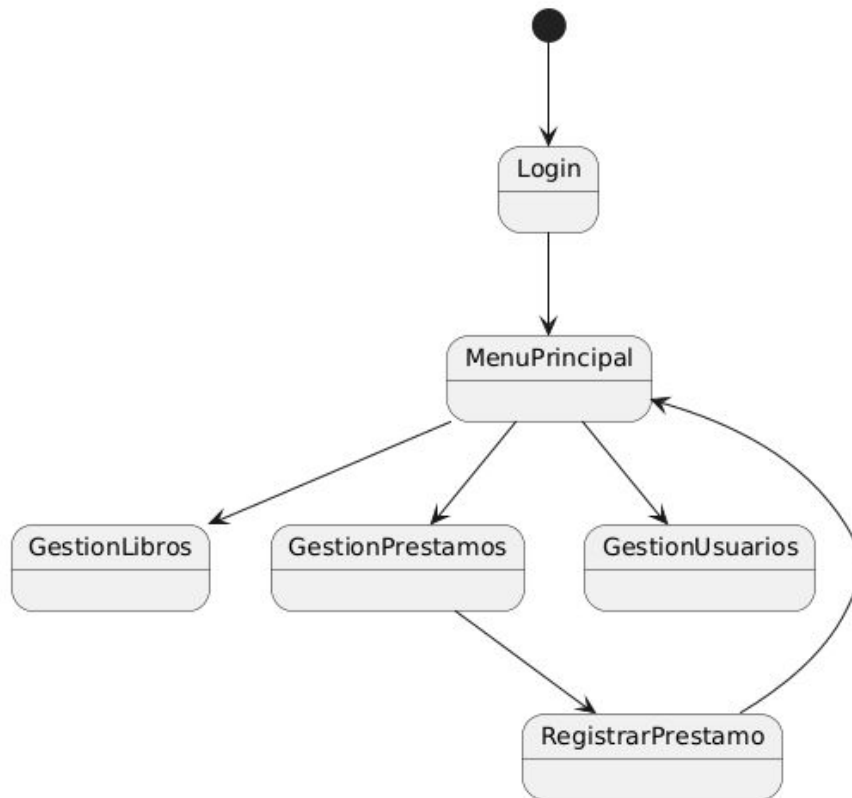


Diagrama de Casos de Uso

Definición

Según **Sommerville (2016)**, un diagrama de casos de uso describe las interacciones entre los actores externos y el sistema, mostrando los servicios que este proporciona.

Pressman (2014) indica que los casos de uso capturan requisitos funcionales desde la perspectiva del usuario.

En el contexto de **diagramas de casos de uso**, cuando hablamos de **interacciones**, nos referimos a las interacciones entre actores y el sistema para lograr un objetivo, y esas interacciones representan **requisitos funcionales**.

Diagrama de Casos de Uso

Categorías

- Primarios: representan objetivos principales del actor.
- Secundarios: soporte o comportamiento auxiliar.
- Relaciones:
 - include: comportamiento obligatorio reutilizable.
 - extend: comportamiento opcional.
 - generalización: herencia entre actores o casos.

Diagrama de Casos de Uso

Casos de uso primarios

Representan los objetivos fundamentales que un actor desea alcanzar mediante el sistema. Son los que justifican la existencia del sistema y expresan directamente los requisitos funcionales principales.

Características:

- Generan valor directo al actor.
- Se inician por un actor externo.
- Responden a metas concretas del usuario.
- Constituyen la base para identificar funcionalidades críticas del sistema.

Diagrama de Casos de Uso

Casos de uso primarios

Ejemplo: el actor persigue un objetivo principal.

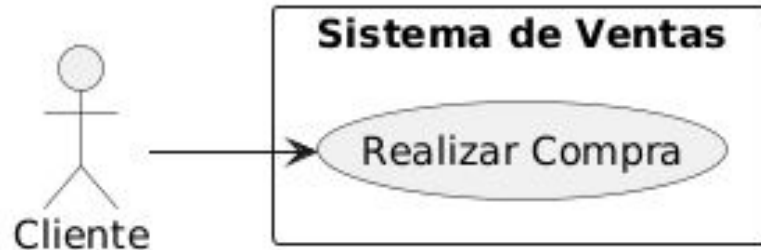


Diagrama de Casos de Uso

Casos de uso secundarios

Representan funcionalidades de soporte que no constituyen por sí mismas un objetivo principal del actor, pero son necesarias para completar un caso de uso primario.

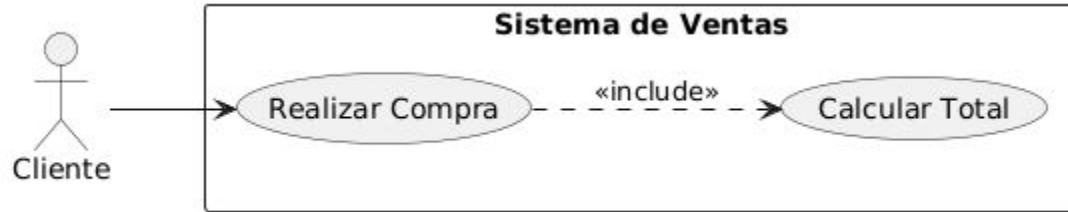
Características:

- No suelen iniciarse directamente por el actor.
- Apoyan o complementan casos principales.
- Pueden reutilizarse en distintos casos de uso.
- Suelen estar vinculados mediante relaciones «include» o «extend».

Diagrama de Casos de Uso

Casos de uso secundarios

Ejemplo: funcionalidad de soporte no iniciada directamente por el actor.



Calcular Total es el caso de uso secundario.

Diagrama de Casos de Uso

Relación «include»

Según **Sommerville (2016)**, la relación «include» indica que un caso de uso incorpora obligatoriamente el comportamiento definido en otro caso de uso.

La relación «include» representa una inclusión obligatoria y reutilizable de comportamiento, donde el caso base depende necesariamente del caso incluido para completarse.

Características:

- El comportamiento incluido siempre se ejecuta.
- Favorece la reutilización.
- Reduce duplicación de lógica.
- Se representa con flecha punteada desde el caso base hacia el caso incluido.

Diagrama de Casos de Uso

Relación «include»

Ejemplo: comportamiento obligatorio reutilizable (al realizar la compra, la validación de pago es obligatoria).

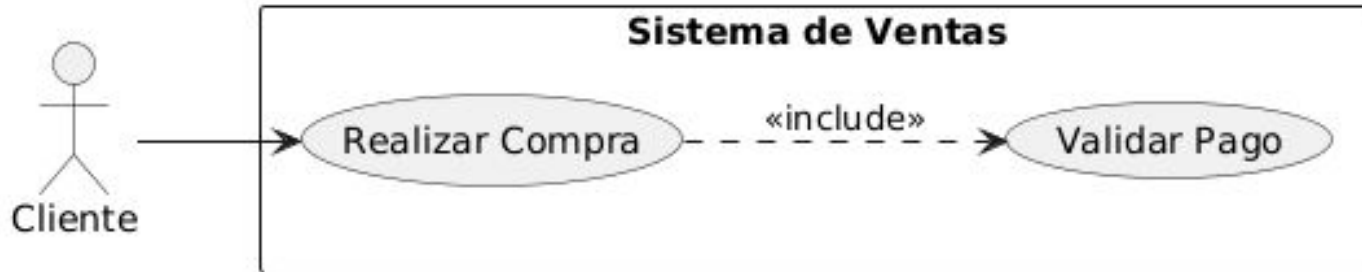


Diagrama de Casos de Uso

Relación «extend»

Pressman (2014) explica que la relación «extend» permite modelar variaciones opcionales dentro de un flujo principal.

La relación «extend» indica que un caso de uso puede ampliar el comportamiento de otro bajo determinadas condiciones, sin que este sea obligatorio.

Características:

- Es opcional.
- Se ejecuta solo si se cumple una condición.
- Permite modelar excepciones o variantes.
- Se representa con flecha punteada desde el caso que extiende hacia el caso base.

Diagrama de Casos de Uso

Relación «extend»

Ejemplo: comportamiento opcional (al realizar la compra, el descuento puede aplicarse o no).

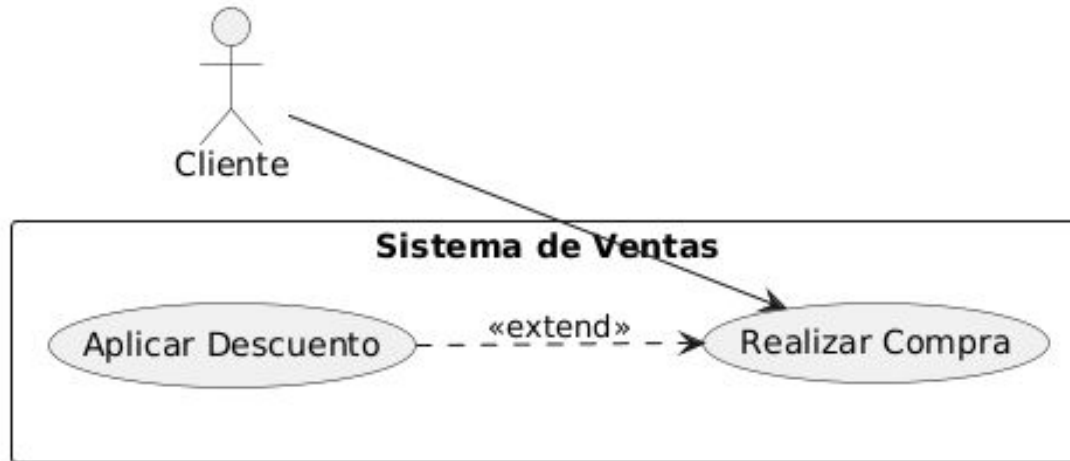


Diagrama de Casos de Uso

Relación de Generalización

Según **Sommerville (2016)**, la generalización en UML implica que un elemento especializado hereda las características de uno más general.

La generalización en casos de uso o actores representa una relación de herencia en la cual el elemento hijo conserva el comportamiento del elemento padre y puede añadir o redefinir comportamientos específicos.

Diagrama de Casos de Uso

Relación de Generalización

a) Actores

Un actor especializado hereda las interacciones del actor general.

Ejemplo conceptual:

- Usuario
 └─ Administrador

b) Casos de uso

Un caso de uso especializado hereda el comportamiento general de otro y puede extenderlo o modificarlo.

Diagrama de Casos de Uso

Relación de Generalización

Características:

- Promueve reutilización conceptual.
- Reduce redundancia en modelos complejos.
- Se representa con línea continua y triángulo hueco apuntando al elemento general.

Diagrama de Casos de Uso

Relación de Generalización

Ejemplo - Generalización de actores:

- Administrador es un tipo especializado de Usuario.
- Administrador hereda todas las interacciones del actor Usuario.
- Administrador puede tener además comportamientos propios adicionales.

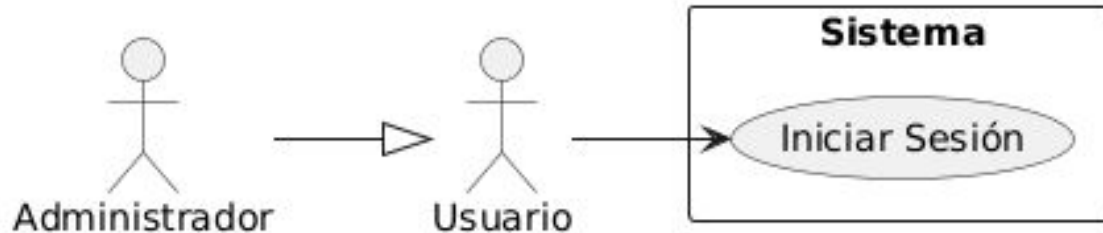


Diagrama de Casos de Uso

Relación de Generalización

Ejemplo - Generalización de casos de uso:

Los pagos específicos heredan el comportamiento general.

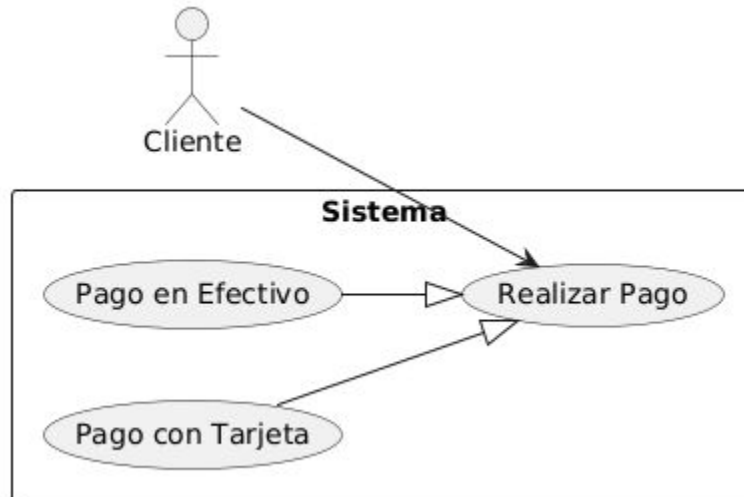


Diagrama de Casos de Uso

Ejemplo aplicado al caso de la Biblioteca:

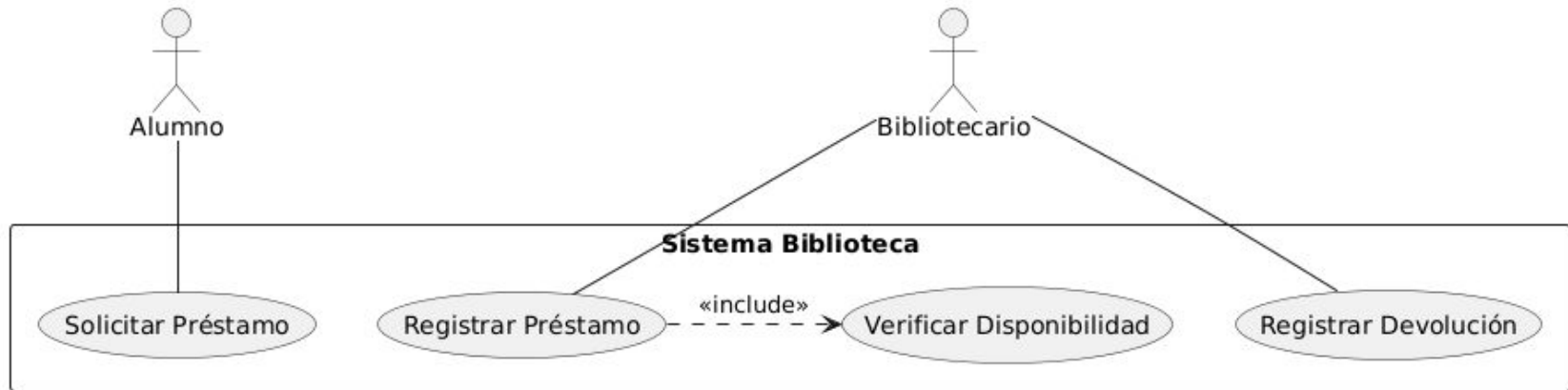


Diagrama de Casos de Uso

Ejemplo aplicado al caso de la Biblioteca

1. Actores

- **Alumno:** actor externo que interactúa con el sistema para solicitar un préstamo. Representa al usuario final que inicia el proceso.
- **Bibliotecario:** actor responsable de registrar préstamos y devoluciones. Interactúa directamente con las funcionalidades administrativas del sistema.

Ambos son entidades externas al sistema, por eso se representan fuera del límite del sistema.

Diagrama de Casos de Uso

Ejemplo aplicado al caso de la Biblioteca

2. Límite del sistema

El rectángulo denominado “**Sistema Biblioteca**” delimita qué funcionalidades pertenecen al sistema.

Todos los casos de uso están contenidos dentro de este límite, lo cual es correcto según la notación UML.

Diagrama de Casos de Uso

Ejemplo aplicado al caso de la Biblioteca

3. Casos de uso

- **Solicitar Préstamo:** objetivo principal del Alumno.
- **Registrar Préstamo:** proceso administrativo ejecutado por el Bibliotecario.
- **Registrar Devolución:** funcionalidad asociada a la gestión del préstamo.
- **Verificar Disponibilidad:** comportamiento necesario para concretar el registro de un préstamo.

Cada caso de uso representa un requisito funcional del sistema.

Diagrama de Casos de Uso

Ejemplo aplicado al caso de la Biblioteca

4. Relación «include»

Es correcta porque:

- Verificar disponibilidad es un paso obligatorio (no puede registrarse un préstamo sin comprobar que el libro esté disponible).
- El comportamiento incluido se ejecuta siempre.
- Representa reutilización de funcionalidad.

No es «extend» porque:

- No es opcional.
- No depende de una condición externa.
- Forma parte del flujo normal del caso base.

En UML, «include» se utiliza cuando el comportamiento es obligatorio; «extend» cuando es condicional u opcional.

Diagrama de Casos de Uso

Ejemplo aplicado al caso de la Biblioteca

5. Ausencia de generalización

No hay generalización entre actores porque:

- Alumno y Bibliotecario no comparten un rol común abstracto modelado en este diagrama.
- No se ha definido un actor general (por ejemplo, “Usuario”) del cual hereden.
- Cada actor cumple funciones distintas sin especialización jerárquica.

La generalización solo se usa cuando existe una relación de herencia clara entre actores o entre casos de uso, lo cual aquí no es necesario para la claridad del modelo.

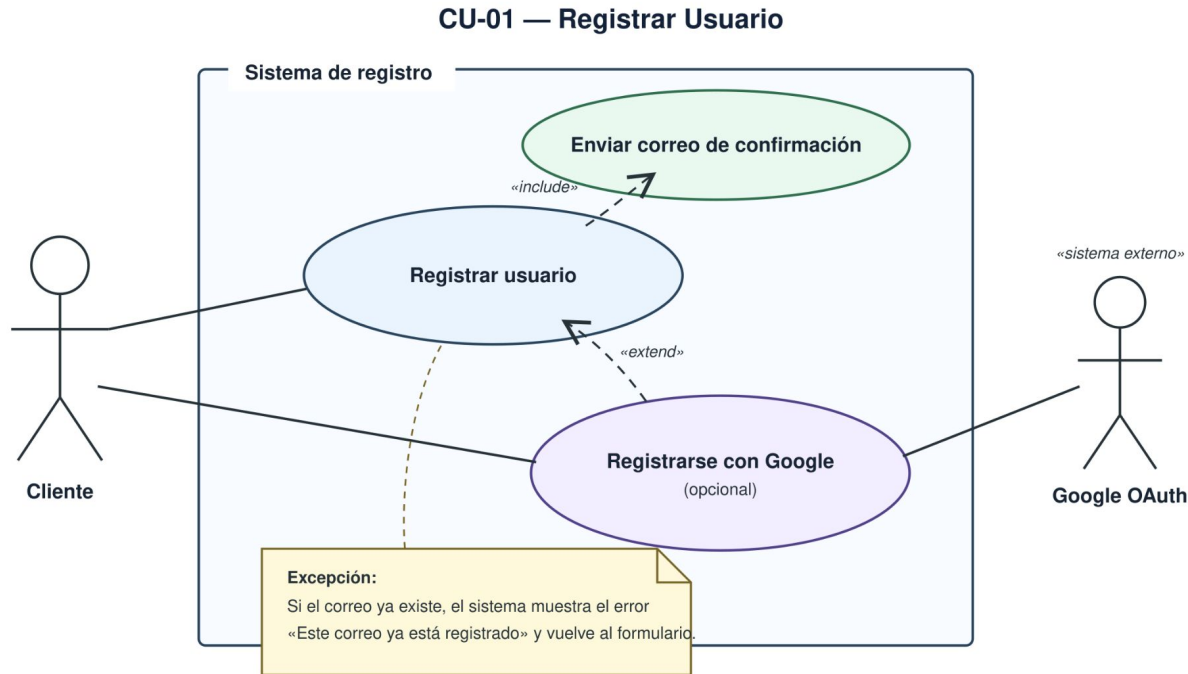
Diagramas de Casos de Uso (ejemplo completo)

Para representar un Caso de Uso (CU) debemos realizarlo en dos partes:

1. Diagrama del CU
2. Especificación (o descripción) del CU

Diagramas de Casos de Uso (ejemplo completo)

1. Diagrama



Diagramas de Casos de Uso (ejemplo completo)

2. Especificación del CU

CASO DE USO: CU-01 Registrar Usuario

=====

ACTOR: Cliente

PRECONDICIÓN: El cliente está en la página de registro.

FLUJO PRINCIPAL:

1. El Cliente completa el formulario con sus datos personales.
2. El Cliente hace clic en "Registrar".
3. El Sistema valida que todos los campos requeridos estén llenos.
4. El Sistema guarda la información en la base de datos.
5. El Sistema envía un correo de confirmación al Cliente.
6. El Sistema muestra el mensaje "Registro exitoso".

FLUJOS ALTERNATIVOS: - En el paso 1, si el Cliente prefiere, puede seleccionar "Registrarse con Google". El Sistema redirige al servicio de autenticación de Google.

FLUJOS DE EXCEPCIÓN: - En el paso 3, si el correo ingresado ya existe:

- a. El Sistema muestra el error "Este correo ya está registrado".
- b. El Sistema vuelve al paso 1 para que el Cliente corrija el dato.

Actores y Relaciones UML

Según **Sommerville (2016)**, un actor representa cualquier entidad externa que interactúa con el sistema.

Las relaciones definen el tipo de interacción entre actores y casos de uso.

Categorías de relaciones:

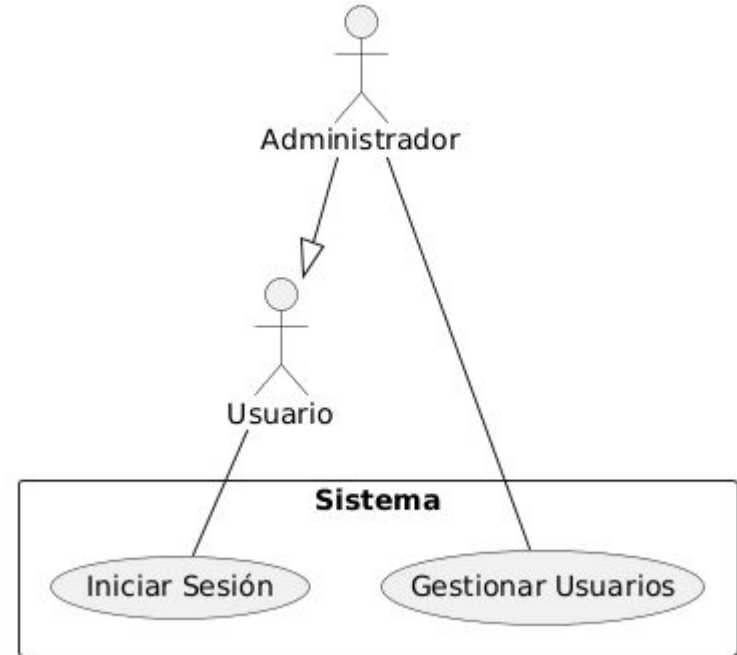
- **Asociación:** vínculo de comunicación.
- **Include:** inclusión obligatoria.
- **Extend:** extensión opcional.
- **Generalización:** especialización entre actores o casos.

Actores y Relaciones UML

Ejemplo: El actor **Administrador** es una especialización del actor **Usuario** mediante una relación de generalización.

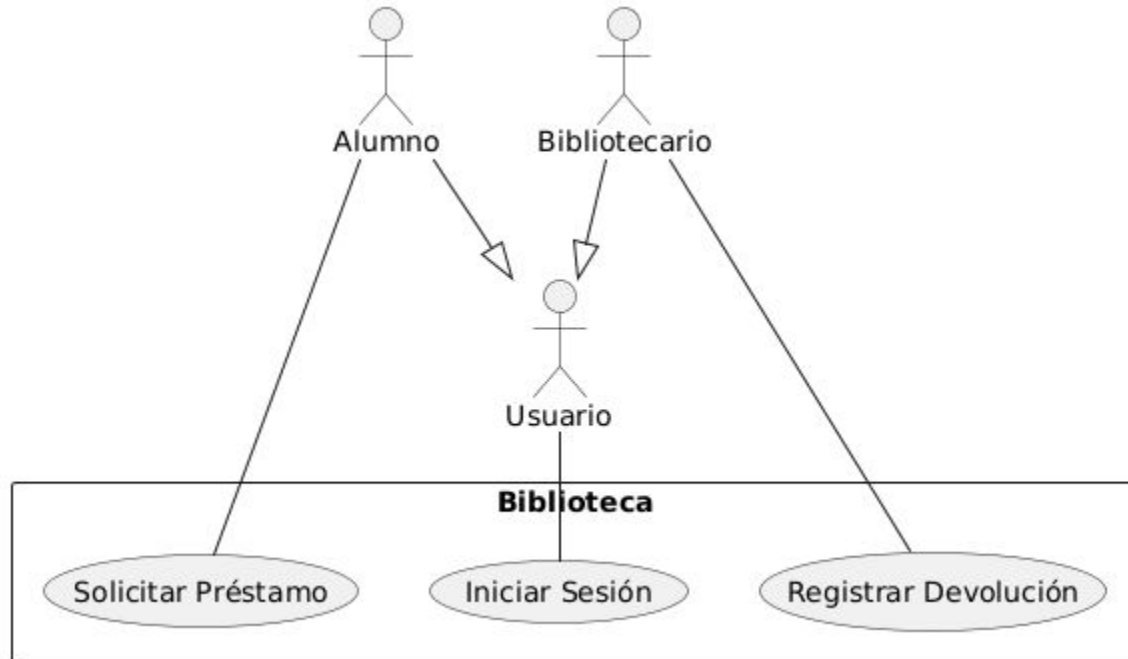
Esto implica que:

- Administrador hereda todas las interacciones asociadas a Usuario.
- Por tanto, Administrador también puede ejecutar el caso de uso *Iniciar Sesión*.
- Además, Administrador incorpora comportamiento específico adicional, representado por el caso de uso *Gestionar Usuarios*.



Actores y Relaciones UML

Retomando el ejemplo de la Biblioteca y aplicando Actores y Relaciones, tenemos:



Actores y Relaciones UML

Retomando el ejemplo de la Biblioteca y aplicando Actores y Relaciones, tenemos:

El actor **Usuario** es general y se especializa en **Alumno** y **Bibliotecario**.

Ambos heredan el caso de uso **Iniciar Sesión**.

Funciones específicas:

- **Alumno** → *Solicitar Préstamo*
- **Bibliotecario** → *Registrar Devolución*

El diagrama modela correctamente requisitos funcionales y utiliza generalización para evitar duplicación.

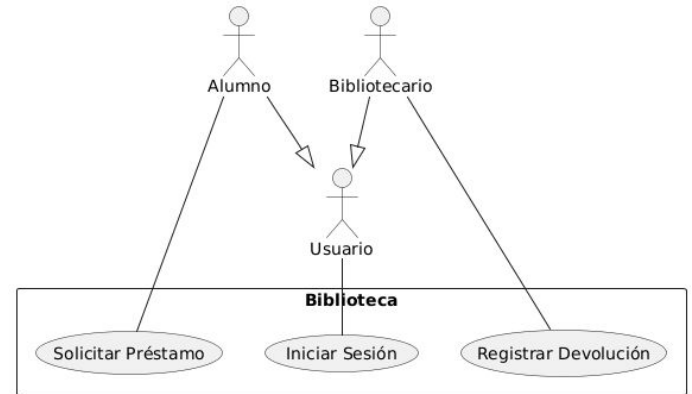


Diagrama de clases

Según **Pressman (2014)**, el diagrama de clases es el principal mecanismo de modelado estructural en UML y muestra clases, atributos, operaciones y relaciones.

Sommerville (2016) señala que representa la estructura estática del sistema.

Clase

Una **clase** representa una abstracción del dominio que define, y contiene:

- Estado (atributos)
- Comportamiento (operaciones)
- Identidad



En UML es la unidad básica del modelado estructural.

Una clase se representa con un rectángulo, y el nombre de la clase en la parte superior.

Diagrama de clases

Estado (atributos)

Un atributo representa una propiedad o característica de la clase.

Forma general: **visibilidad nombre : tipo** (ejemplo: - nombre : String)

La visibilidad puede ser:

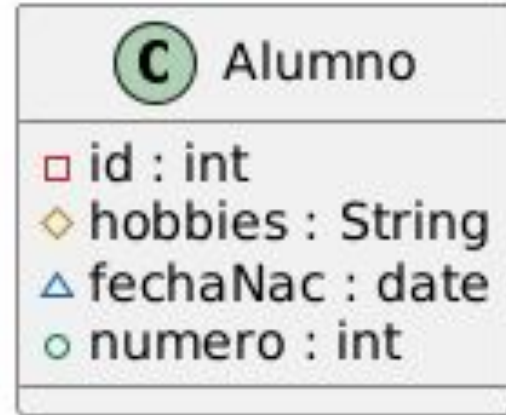
Símbolo	Significado	Equivalente en programación
+	Público	public
-	Privado	private
#	Protegido	protected
~	Paquete (default)	package / default

Diagrama de clases

Atributo

En este ejemplo, el editor utilizado representa la visibilidad mediante figuras geométricas y colores:

- **Círculo verde:** public (+)
- **Cuadrado rojo:** private (-)
- **Rombo amarillo:** protected (#)
- **Triángulo azul:** paquete (~)



Formalmente, nosotros lo representaremos con la nomenclatura +, -, #, ~.

Diagrama de clases

Operación (método en programación)

Una **operación** define el comportamiento que la clase ofrece.

Forma general: **visibilidad nombre(parámetros) : tipoRetorno**

Ejemplo: **+ getNumero() : void**

En programación se implementa como método.

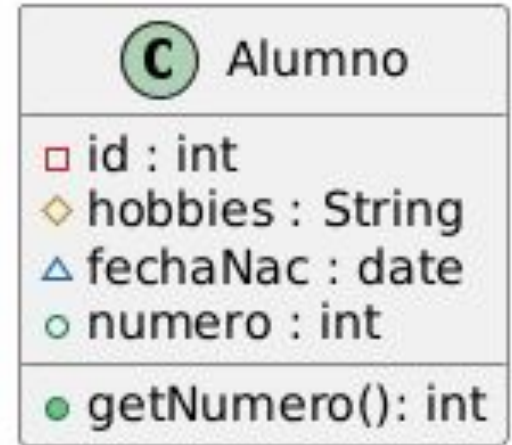


Diagrama de clases

Identidad

La **identidad** es la propiedad que permite distinguir una instancia de otra, incluso cuando ambas tienen el mismo estado.

Dos objetos pueden tener los mismos valores en sus atributos, pero siguen siendo objetos distintos porque poseen identidad propia.

Ejemplo: Supongamos dos objetos Alumno:

id	Nombre
15	Juan Pérez
22	Juan Pérez

Aunque tengan los mismos valores, son instancias diferentes en memoria. Cada una tiene identidad propia, con lo cual no es necesario modelarla explícitamente.

Diagrama de clases

Identidad

Nota:

Los atributos que en el modelo relacional podrían definirse como PK, UK o FK se modelan en UML simplemente como atributos de la clase, sin indicar explícitamente su rol de clave.

En el ejemplo, “id” es un atributo más del dominio.

Su función como clave primaria corresponde al diseño de base de datos, no al modelo UML conceptual.

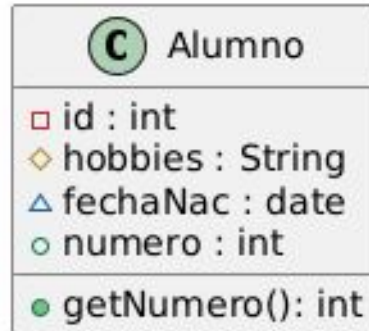


Diagrama de clases

Relaciones entre clases

- **Asociación:** relación estructural entre clases.
- **Agregación:** relación “todo-parte” débil.
- **Composición:** relación “todo-parte” fuerte.
- **Herencia:** especialización de clase.

Diagrama de clases

1. Asociación

Es una **relación estructural básica** que indica que dos clases están vinculadas porque sus objetos interactúan o se conocen.

- Representa colaboración o comunicación: Una asociación indica que los objetos de una clase **interactúan o colaboran** con objetos de otra clase para cumplir una responsabilidad del sistema. No implica pertenencia, sino vínculo funcional.
- Puede tener multiplicidad (1, 0..*, etc.): La multiplicidad indica **cuántas instancias** de una clase pueden estar relacionadas con una instancia de la otra. Se expresa mediante cardinalidades en cada extremo de la asociación.
- No implica dependencia de ciclo de vida: El ciclo de vida de un objeto es el período en el que existe (desde su creación hasta su destrucción). En una asociación simple, los objetos pueden existir independientemente; la destrucción de uno no obliga a destruir el otro.
- Es la relación más general entre clases.

Diagrama de clases

1. Asociación

Ejemplo: Un *Alumno* realiza un *Préstamo*.

Para determinar si es una Asociación, nos preguntamos:

¿Existe simplemente una relación o vínculo entre las clases, sin que una sea parte estructural de la otra?

Si la respuesta es sí, estamos ante una **Asociación**.

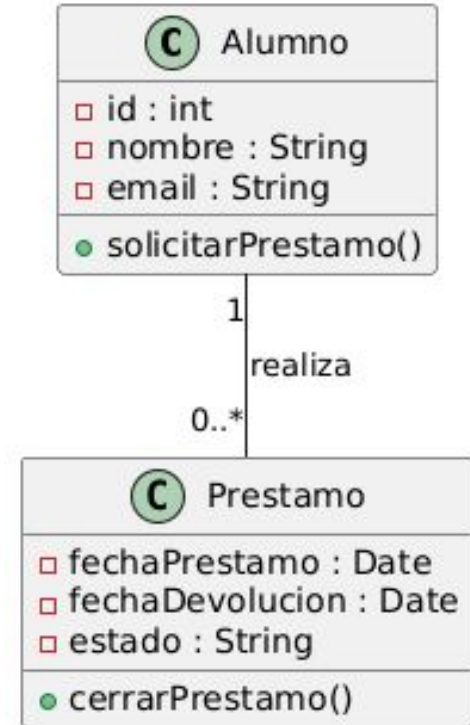


Diagrama de clases

2. Agregación

Es una forma especial de asociación que representa una relación **todo–parte débil**.

- La parte puede existir independientemente del todo.
- Si el todo se elimina, las partes pueden seguir existiendo.
- Se representa con un rombo blanco en el extremo del “todo”.

En UML, una relación todo–parte débil (agregación) significa:

Existe una relación de pertenencia conceptual, pero no hay dependencia de ciclo de vida entre el todo y la parte.

Ejemplo: Una *Biblioteca* contiene *Libros*, pero los libros pueden existir fuera de ella.

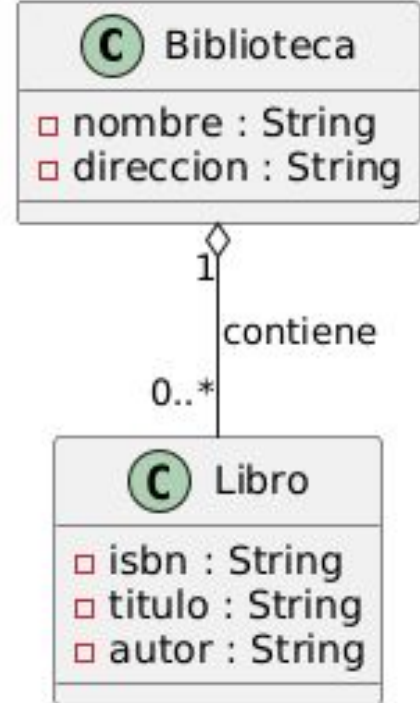


Diagrama de clases

2. Agregación

¿Quién es “débil”? ¿El todo o la parte?

En agregación UML:

- La parte es independiente.
- Lo que es “débil” es el vínculo.

No es que el todo o la parte sean débiles como entidad.

La debilidad se refiere a la fuerza de la relación.

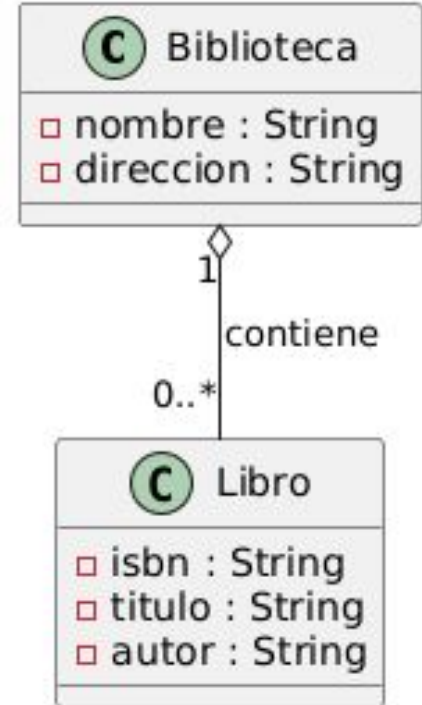


Diagrama de clases

2. Agregación

Explicación del diagrama

- $o--\rightarrow$ representa **agregación** (rombo blanco).
- El rombo se ubica del lado del “todo” (**Biblioteca**).
- Multiplicidad:
 - Una Biblioteca puede contener cero o muchos Libros.
 - Cada Libro puede estar asociado a una Biblioteca (según el modelo).

Esto indica relación todo–parte débil: si se elimina la Biblioteca, los Libros pueden seguir existiendo.

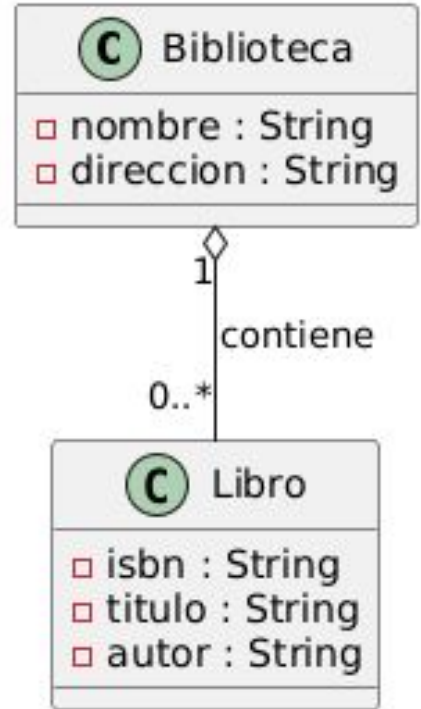


Diagrama de clases

2. Agregación

Para determinar si es agregación preguntamos:

¿Existe relación todo–parte donde la parte puede existir sin el todo?

Si la respuesta es sí, estamos ante una **Agregación**.

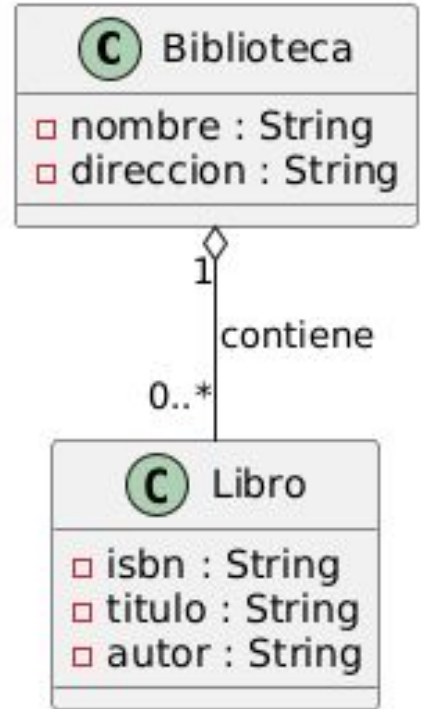


Diagrama de clases

3. Composición

Es una relación **todo–parte fuerte**, donde existe dependencia de ciclo de vida.

- La parte no puede existir sin el todo.
- Si el todo se elimina, las partes también se eliminan.
- Se representa con un rombo negro en el extremo del “todo”.

Ejemplo:

Un *Préstamo* está compuesto por *Detalles de Préstamo*.

Si se elimina el préstamo, los detalles desaparecen.

Diagrama de clases

4. Herencia (Generalización)

Es una relación de especialización donde una clase hija hereda atributos y métodos de una clase padre.

- Representa una relación “es un”.
- Permite reutilización y extensión de comportamiento.
- Se representa con línea continua y triángulo hueco hacia la clase general.

Ejemplo: *Alumno* es un tipo de *Usuario*.

Diagrama de clases

Relaciones - comparación

Relación	Tipo de vínculo	Dependencia de vida	Notación UML	Ejemplo
Asociación	Relación estructural general	No	Línea simple	Alumno — Préstamo
Agregación	Todo–parte débil	No	Rombo blanco	Biblioteca ◇— Libro
Composición	Todo–parte fuerte	Sí	Rombo negro	Préstamo ◆— Detalle
Herencia	Especialización (“es un”)	No aplica	Triángulo hueco	Alumno ▷ Usuario

Conceptos fundamentales: Clases, atributos, métodos y relaciones

Según **Sommerville (2016)**, el diagrama de clases describe la estructura estática del sistema, representando las clases, sus atributos, operaciones y las relaciones entre ellas.

Clase

Una **clase** es una abstracción que representa un conjunto de objetos con características y comportamiento comunes.

Define:

- Estado (atributos)
- Comportamiento (operaciones)
- Identidad

Conceptos fundamentales: Clases, atributos, métodos y relaciones

Atributos

Un **atributo** representa una propiedad o característica de la clase.

Forma general: visibilidad nombre : tipo

Ejemplo: - **nombre** : **String**

Operaciones (métodos)

Una **operación** define el comportamiento que la clase ofrece.

Forma general: visibilidad nombre(parámetros) : tipoRetorno

Ejemplo: + **registrarPrestamo()** : **void**

Conceptos fundamentales: Clases, atributos, métodos y relaciones

Relaciones entre clases

Asociación

Relación estructural general entre clases. No implica dependencia de ciclo de vida.

Agregación

Relación todo–parte débil. La parte puede existir sin el todo.

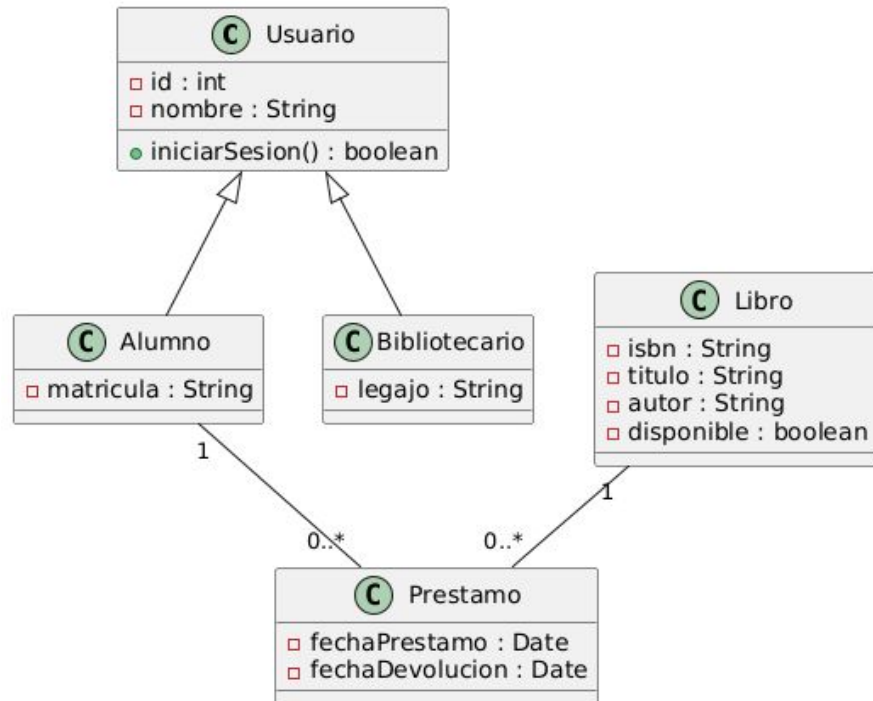
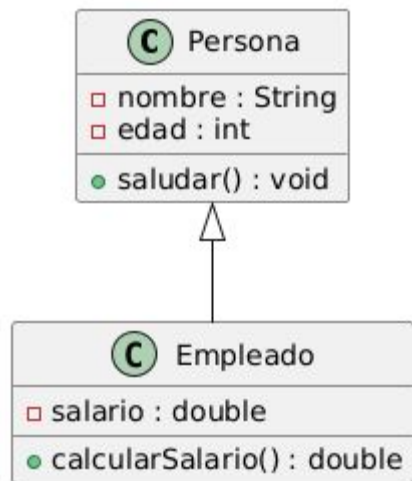
Composición

Relación todo–parte fuerte. La parte depende del ciclo de vida del todo.

Herencia (generalización)

Relación de especialización donde una clase hija hereda atributos y operaciones de una clase padre.

Ejemplos



Diagramas de Secuencia

Según Sommerville (2016), un diagrama de secuencia representa cómo los objetos interactúan entre sí a lo largo del tiempo mediante el intercambio de mensajes.

Pressman (2014) indica que este tipo de diagrama permite modelar la dinámica del sistema mostrando el orden en que ocurren las interacciones.

Diagramas de Secuencia

Elementos básicos

Objeto

Representa una instancia que participa en la interacción.
Se ubica en la parte superior del diagrama.

Ejemplo:

Alumno

Sistema

Base de Datos

Diagramas de Secuencia

Elementos básicos

Línea de vida

Es una línea vertical que representa la existencia del objeto durante la interacción.

Diagramas de Secuencia

Elementos básicos

Mensaje

Es una comunicación entre objetos.

Se representa con una flecha.

Puede ser:

- Solicitud
- Respuesta

Ejemplo:

Solicitar préstamo

Confirmar préstamo

Diagramas de Secuencia

Elementos básicos

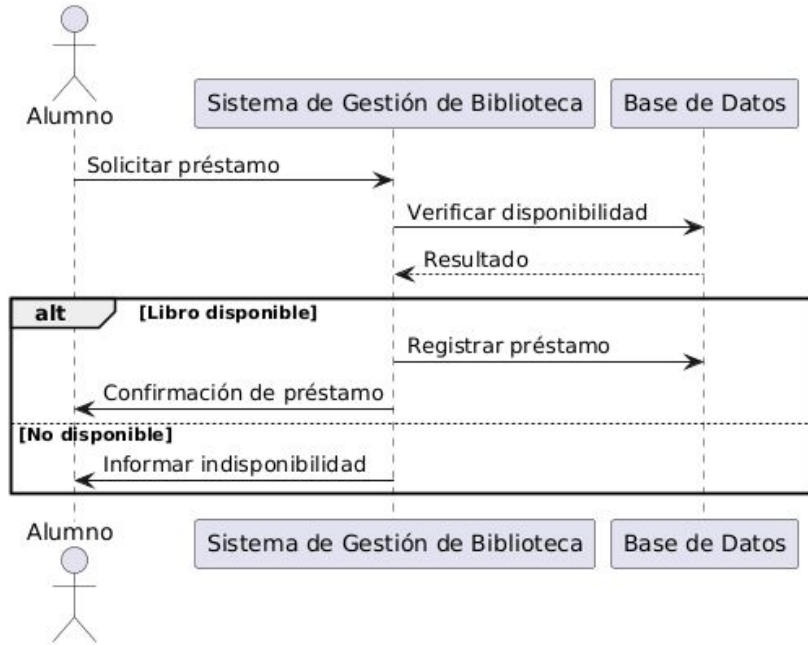
Orden temporal

El tiempo avanza de arriba hacia abajo.

Diagramas de Secuencia

Ejemplo: Sistema de Gestión de Biblioteca — Solicitud de Préstamo

Diagrama de Secuencia - Solicitud de Préstamo



Interpretación del diagrama

1. El alumno solicita un préstamo.
2. El sistema consulta la base de datos.
3. La base de datos responde.
4. El sistema decide:
 - Si (alt) está disponible → registra y confirma.
 - Si no → informa indisponibilidad.

Nota: “alt” indica que el flujo del sistema tiene **ramas alternativas condicionadas**, y se usa para representar decisiones dentro del diagrama de secuencia.

Referencias

Pressman, R. S. (2014). Ingeniería del software: Un enfoque práctico. McGraw-Hill.

Sommerville, I. (2016). Ingeniería del software. Pearson.

Programa Ingeniería del Software – DGETP